

FPT UNIVERSITY

Capstone Project Document

Lightweight Deep Learning for Robust Shrimp Disease
Detection on Mobile Devices under Diverse Environmental
Conditions

CVio	
Group Members	Tran Huu Nhan (Leader) - CE181655 Le Thi Huynh Nhu - CA182007 Nguyen Van Phong - CE192081
Supervisor	Nguyen Dinh Vinh
Ext Supervisor	N/A
Capstone Project code	AIP491_01

Table of Contents

Acknowledgement	ii
Definition and Acronyms	iii
List of tables	iv
List of figures	v
IV. Methodology	1
1. Methodological Framework and Experimental Scope	1
2. Classification Methodology	1
2.1 Classification Image Preparation and Fixed Experimental Protocol	1
2.2 Candidate-Model Screening Procedure	2
2.3 Proposed YOLO26m-cls Classification Architecture	3
2.4 SimAM-DCFR Feature Recalibration	3
2.5 Custom Single-Label ASL-LDAM Objective	5
2.6 Classification Training and Evaluation Protocol	7
2.6.1 Multi-Source Retraining Protocol	7
3. Instance-Segmentation Methodology	7
3.1 Disease-Region Annotation Protocol and Label Semantics	7
3.2 Leakage-Aware Specimen-Grouped Partitioning	9
3.2.1 Original and Expanded Instance-Segmentation Dataset Construction	10
3.3 YOLO11n-seg Reference Architecture and Augmentation Controls	11
3.4 Attention-Based Multi-Scale Feature Recalibration	13
3.4.1 Coordinate Attention Followed by SimAM	13
3.4.2 LKA-to-SimAM	14
3.4.3 Dual-Polarity Contrast Attention	14
3.4.4 CoTEGate and BoundaryLite	16
3.5 Attention Efficiency and Placement-Control Protocol	17
4. Fourier-Domain Segmentation Methodology	18
4.1 Frequency-Domain Formulation	18
4.2 Image-Level Fourier Candidate Families	18
4.3 W1 High-Pass Reference Configuration	20
4.4 Paired Image-Spectrum Interaction Protocol	23
4.5 Feature-Domain Fourier Refinement	23
4.6 Fourier and Augmentation Order Controls	24
4.7 Segmentation Corruption and Runtime Protocol	24
5. Evaluation Metrics and Decision Rules	24
5.1 Classification Metrics	24
5.2 Instance-Segmentation Metrics	25
5.3 Confidence, NMS, and Checkpoint Rules	25
6. Mobile Runtime and Real-World Acquisition Protocol	26
6.1 Common Android Application and Conditional Two-Stage Execution	26
6.2 Three-Camera Real-World Acquisition Set	28
6.3 Runtime Device Configuration and Sampling	30
6.4 Interpretation Boundary for the Mobile Experiment	30
7. Experimental Environment and Reproducibility	30
8. Methodological Validity and Scope Boundaries	31
References	33

Acknowledgement

The team sincerely thanks Nguyen Dinh Vinh, Lecturer and Supervisor, for his academic guidance, milestone reviews, and detailed feedback throughout the capstone project. We acknowledge FPT University Can Tho for the academic environment and project framework. We also acknowledge the authors and maintainers of the public shrimp-image datasets, open-source frameworks, and research tools used in the classification, instance-segmentation, robustness, explainability, and mobile application work. Finally, the team thanks the lecturers, colleagues, friends, and families who supported experimentation, application development, documentation, presentation, and defense preparation.

Definition and Acronyms

The following terms are used throughout this standalone report. The list is limited to terminology that appears in Report 4.

Acronym	Definition
AMP	Automatic Mixed Precision
AP	Average Precision
ASL	Asymmetric Loss
ASL-LDAM	Custom single-label objective combining ASL-style asymmetric focusing with Label-Distribution-Aware Margin
BCE	Binary Cross-Entropy
BG	Black Gill disease
CA	Coordinate Attention
CAM	Class Activation Mapping
CoTEGate	Project-created consensus-based residual feature-recalibration gate
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DCFR	Project-created texture feature-recalibration component used with SimAM
DFT	Discrete Fourier Transform
DPCA	Dual-Polarity Contrast Attention
EigenCAM	Principal-component-based class activation mapping method
FFT	Fast Fourier Transform
FPS	Frames Per Second
GAP	Global Average Pooling
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
Grad-CAM++	Generalized gradient-based class activation mapping
HiResCAM	High-Resolution Class Activation Mapping
HScore	Project-defined healthy-aware instance-segmentation decision score
HSV	Hue, Saturation, and Value color representation
IoU	Intersection over Union
LDAM	Label-Distribution-Aware Margin
LKA	Large Kernel Attention
mAP50	Mean Average Precision at an IoU threshold of 0.50
mAP50:95	Mean Average Precision over IoU thresholds 0.50 to 0.95
NMS	Non-Maximum Suppression
PAN-FPN	Path Aggregation Network and Feature Pyramid Network
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RGB	Red, Green, and Blue color representation
SimAM	Simple, parameter-free attention module based on neuron energy
TFLite	TensorFlow Lite
TIMM	PyTorch Image Models library
WSSV	White Spot Syndrome Virus
WSSV_BG	Four-class image-level co-infection category containing BG and WSSV evidence
YOLO	You Only Look Once model family

List of tables

Table 4.1. Classification preprocessing and fixed experimental controls	2
Table 4.2. ASL-LDAM and SimAM-DCFR implementation constants	5
Table 4.3. Classification training configuration	7
Table 4.4. Segmentation annotation rules and label semantics	9
Table 4.4A. Recognized specimen-group subset under the nominal 80/10/10 grouped protocol	10
Table 4.4B. Expanded instance-segmentation image inventory	10
Table 4.5. Segmentation split-policy definitions	11
Table 4.6. Segmentation augmentation and control policies	13
Table 4.7. Recorded attention configuration protocols	17
Table 4.8. Attention implementation and efficiency metadata	18
Table 4.9. Fourier candidate families and recorded sweep settings	20
Table 4.10. Fourier-augmentation processing-order protocols	24
Table 4.11. Evaluation thresholds and checkpoint rules	26
Table 4.12. Common on-device application and model configuration	28
Table 4.13. Android runtime device configuration	30
Table 4.14. Training, analysis, and reproducibility configuration	30

List of figures

Figure 4.1. Classification experimental workflow	2
Figure 4.2. YOLO26m-cls architecture with SimAM-DCFR and ASL-LDAM	3
Figure 4.3. Module-level SimAM-DCFR computation	5
Figure 4.4. Training-time ASL-LDAM computation graph	6
Figure 4.5. Representative project-authored BG and WSSV disease-region annotations	8
Figure 4.6. Comparison of segmentation split policies	11
Figure 4.7. YOLO11n-seg attention integration	14
Figure 4.8. Dual-Polarity Contrast Attention workflow	16
Figure 4.9. CoTEGate and BoundaryLite integration	17
Figure 4.10. Representative W1 Fourier workflow	22
Figure 4.11. Exact W1 spatial/frequency decomposition	22
Figure 4.12. Matched five-path W1-augmentation spectrum study	23

IV. Methodology

1. Methodological Framework and Experimental Scope

This study was organized around two complementary computer-vision tasks. The first task was four-class image classification, in which each input image was assigned to Healthy, Black Gill disease (BG), White Spot Syndrome Virus (WSSV), or the WSSV_BG co-infection category. The second task was disease-region instance segmentation, in which visible BG and WSSV regions were represented by instance masks. Classification provided a compact image-level decision suitable for mobile-oriented screening, whereas instance segmentation provided spatial evidence for inspecting localized symptoms and evaluation leakage.

Three controlled experimental branches were maintained. The classification branch compared candidate backbones under a fixed seed-42 image partition, then trained a YOLO26m-cls classifier with the proposed SimAM-DCFR feature recalibration and the training-only ASL-LDAM objective. The attention-based segmentation branch inserted feature-recalibration modules immediately before the YOLO11n-seg Segment head. The Fourier branch applied image-space frequency transformations, and separately considered feature-space frequency refinement, without conflating either branch with the classification method. Model conversion and runtime measurements were treated as downstream feasibility checks; mobile interface behavior is described in Chapter V rather than in this chapter.

The methodology separated training operations from inference operations. Background removal, augmentation, feature recalibration, custom losses, checkpoint selection, corruption generation, and explanation-map generation were recorded as distinct stages. Performance outcomes, model rankings, confusion matrices, corruption gains, and comparative conclusions are reserved for Chapter VI.

2. Classification Methodology

2.1 Classification Image Preparation and Fixed Experimental Protocol

The classification source contained four image-level categories. Each file was loaded as an RGB image, processed through an offline foreground-extraction stage, and written to a standardized directory tree while preserving filename identity. The background-removal stage was completed before model training so that directly compared classifiers received the same stored image representation. A class-order check enforced the serialized order Healthy, BG, WSSV, and WSSV_BG in manifests.

A fixed seed-42 manifest targeted a 70/15/15 train/validation/test allocation for classification. The retained four-class manifest contained 804 training, 172 validation, and 173 test images, with the serialized class order Healthy, BG, WSSV, and WSSV_BG preserved in every split. All directly compared classification candidates reused exactly the same partition membership. Training samples were resized or cropped to 224×224 , converted to tensors, normalized, and augmented with the retained RandAugment configuration; validation and test used deterministic preprocessing. This classification split is distinct from the 80/10/10 grouped protocol used for instance segmentation.

Table 4.1: Classification preprocessing and fixed experimental controls

Stage	Executed operation	Control for direct comparison
Source loading	RGB decoding with filename preservation	Identical source inventory and class order
Offline preparation	Background removal followed by standardized image storage	One processed image copy reused by all directly compared classifiers
Partitioning	Fixed seed-42 nominal 70/15/15 train/validation/test manifest	Partition membership was not regenerated per model
Training input	Resize/crop to 224×224 ; tensor conversion; normalization; configured RandAugment	Stochastic transformations applied only to training samples
Validation/test input	Deterministic resize and normalization	No random crop or stochastic augmentation
Traceability	Stored manifest, class mapping, configuration, checkpoint, and prediction files	File identity remained recoverable across stages

Table 4.1 defines the controls used to isolate differences in model family, loss, and feature recalibration. Detailed class distributions and dataset-side descriptive statistics are reported with the experimental results in Chapter VI.

As shown in Figure 4.1, the classification workflow proceeded from offline preparation and a fixed protocol to candidate screening, selected-model training, and four evaluation branches. The figure distinguishes the preprocessing stored on disk from transformations applied only during training.

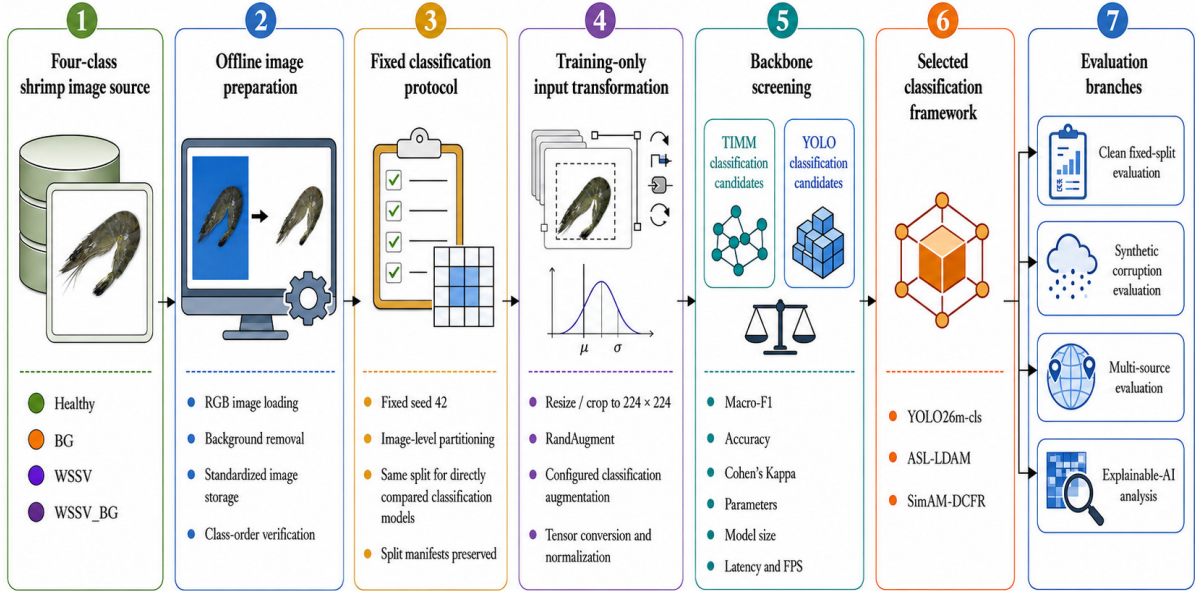


Fig. 4.1. Classification experimental workflow from four-class image preparation and fixed protocol control to candidate-model screening, the selected classification framework, and four evaluation branches.

2.2 Candidate-Model Screening Procedure

Candidate screening covered 17 classifiers from the TIMM model collection and 15 official Ultralytics classification variants spanning YOLOv8, YOLO11, and YOLO26 families. All candidates received the

fixed processed dataset, the same class order, and the same seed-42 partition. The screening comparison used Macro-F1 as the primary predictive criterion alongside accuracy, Cohen’s kappa, parameter count, checkpoint size, measured latency, and FPS. For final mobile-oriented selection, the deployment preference was approximately 15 million parameters or fewer. Larger models retained in the historical screen are higher-capacity diagnostic controls, not eligible final mobile candidates. YOLO26x-cla (28.36M parameters) is therefore a capacity control outside the final deployment-oriented envelope. This threshold is project-specific. The historical screen used test-partition Macro-F1 as a selection signal. Consequently, that partition was inspected during candidate selection and cannot be interpreted as a completely untouched final test set for the screening stage. The procedure is reported as executed rather than retrospectively rewritten as validation-only selection. Subsequent training retained the validation-selected checkpoint and used held-out evaluation for the selected configuration, but the screening limitation remains relevant when interpreting model-family comparisons.

The complete TIMM candidate benchmark and the retained Ultralytics comparison are reported in Chapter VI. The role of this subsection is to define the common protocol and the decision criteria, not to rank candidates within the methodology chapter.

2.3 Proposed YOLO26m-cla Classification Architecture

The selected reference classifier was YOLO26m-cla. The original feature extractor and Ultralytics Classify head were retained, while the proposed SimAM-DCFR module was inserted between the terminal feature tensor and the classification head. The module accepted a tensor $F \in \mathbb{R}^{B \times C \times H \times W}$, preserved its shape, and returned a recalibrated tensor F' to the unchanged head. The head produced four logits $z \in \mathbb{R}^{B \times 4}$ in the fixed class order.

The custom ASL-LDAM objective was applied only during training. At inference, the model produced logits and selected the class index by $\arg \max_k z_k$; neither the LDAM margin nor the loss-side weighting was required in the exported inference graph. Figure 4.2 shows this separation and the placement of the proposed feature-recalibration module.

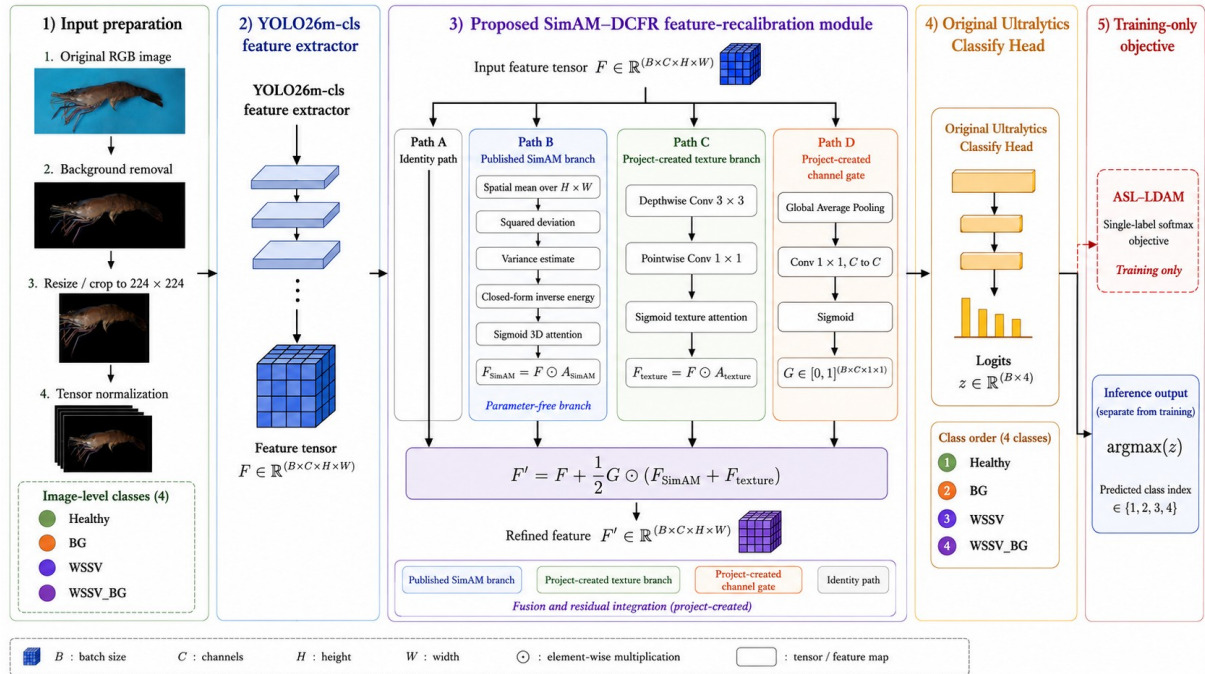


Fig. 4.2. YOLO26m-cla architecture with the proposed SimAM-DCFR feature-recalibration module, the original classification head, and the training-only ASL-LDAM objective.

2.4 SimAM-DCFR Feature Recalibration

The SimAM-DCFR module combined four parallel paths from the same input tensor: an identity path, the published SimAM branch [1], a project-created texture-recalibration branch, and a project-created

channel gate. Only the SimAM branch was parameter-free. The texture branch and channel gate contained trainable convolutions.

For each channel, SimAM first computed the spatial mean

$$\mu = \text{mean}_{H,W}(F), \quad (4.1)$$

and the squared deviation

$$D = (F - \mu)^2. \quad (4.2)$$

The spatial variance estimate was

$$V = \frac{\sum_{H,W} D}{\max(1, HW - 1)}. \quad (4.3)$$

Using $\lambda = 10^{-4}$, the inverse-energy response and attention map were defined as

$$E_{inv} = \frac{D}{4(V + \lambda)} + \frac{1}{2}, \quad A_{SimAM} = \sigma(E_{inv}), \quad (4.4)$$

which yielded

$$F_{SimAM} = F \odot A_{SimAM}. \quad (4.5)$$

The texture branch used a channel-wise depthwise convolution followed by pointwise projection:

$$A_{texture} = \sigma[PWConv_{1 \times 1}(DWConv_{3 \times 3}(F))], \quad (4.6)$$

$$F_{texture} = F \odot A_{texture}. \quad (4.7)$$

The depthwise 3×3 convolution used C groups and no bias; the pointwise 1×1 convolution also used no bias. This path learned local texture responses while preserving the input dimensions.

The channel gate summarized each channel by global average pooling and projected the summary through a trainable 1×1 convolution:

$$G = \sigma[Conv_{1 \times 1}(GAP(F))], \quad G \in [0, 1]^{B \times C \times 1 \times 1}. \quad (4.8)$$

The gate was spatially broadcast during fusion. The branch average and residual output were

$$F_{branch} = \frac{1}{2}(F_{SimAM} + F_{texture}), \quad (4.9)$$

$$F' = F + G \odot F_{branch} = F + \frac{1}{2}G \odot (F_{SimAM} + F_{texture}). \quad (4.10)$$

Figure 4.3 maps the equations above to the four-path computation. The identity connection maintains the original representation, while the channel gate controls the contribution of the two recalibrated feature paths.

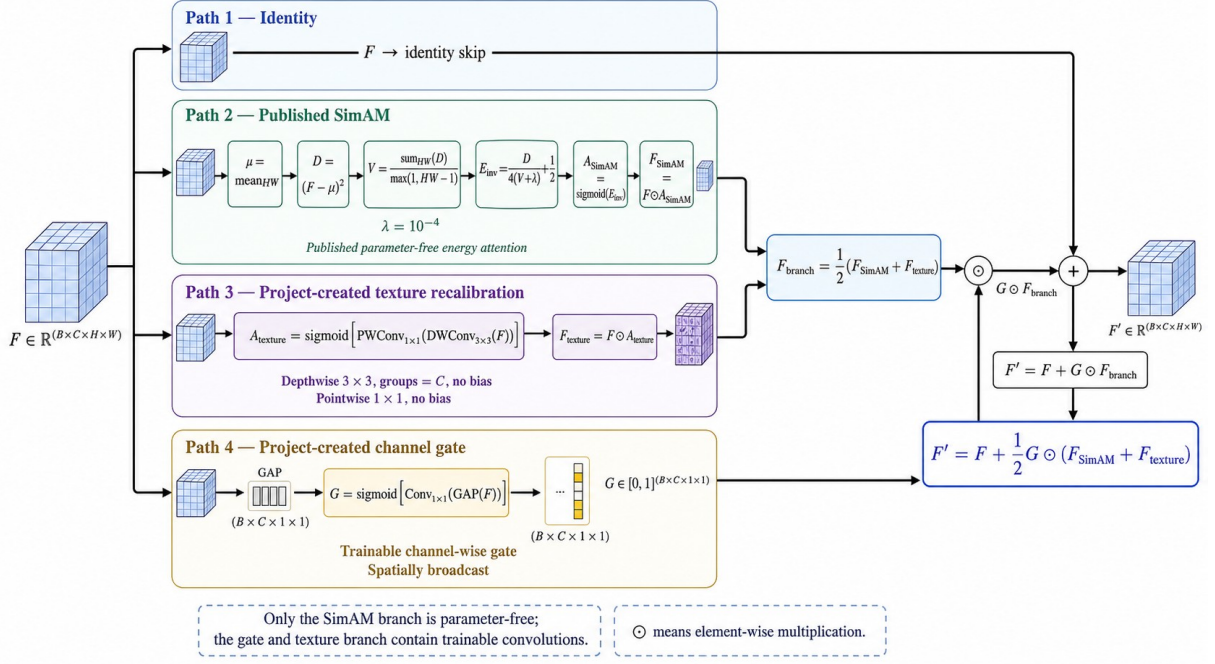


Fig. 4.3. Module-level computation of SimAM-DCFR, showing the identity path, published SimAM branch, project-created texture branch, trainable channel gate, and residual fusion.

The implementation constants that define the module and the custom objective are consolidated in Table 4.2.

Table 4.2: ASL-LDAM and SimAM-DCFR implementation constants

Component	Recorded value	Methodological role
SimAM en- ergy constant	$\lambda = 10^{-4}$	Stabilizes the variance-dependent inverse-energy expression
Texture branch	DWConv 3×3 , groups = C , no bias; PW-Conv 1×1 , no bias	Learns spatial texture attention without changing tensor shape
Channel gate	GAP; Conv 1×1 , C to C ; sigmoid	Produces a trainable channel-wise residual gate
LDAM	$m_{\max} = 0.5$, scale $s = 30$	Applies class-dependent true-class margins before softmax
Training class counts	$[282, 139, 229, 154]$	Defines the class-dependent LDAM margins in fixed class order

Table 4.2 contains method-defining constants rather than measured outcomes. The class counts are included because they enter the LDAM margin construction directly.

2.5 Custom Single-Label ASL-LDAM Objective

The custom objective combined label-distribution-aware margins [2] with asymmetric focusing inspired by ASL [3], but it was implemented for single-label softmax classification rather than sigmoid one-versus-rest prediction. Let $z \in \mathbb{R}^{B \times K}$ denote the logits, y_i the class-index target, and $K = 4$. The class-index target was converted to one-hot form:

$$q_{ik} = \mathbf{1}[k = y_i]. \quad (4.11)$$

For class count n_k , the LDAM radius and normalized margin were

$$r_k = n_k^{-1/4}, \quad m_k = m_{\max} \frac{r_k}{\max_j r_j}, \quad m_{\max} = 0.5. \quad (4.12)$$

Only the true-class logit received the margin, after which all logits were scaled:

$$\tilde{z}_{ik} = s(z_{ik} - q_{ik}m_k), \quad s = 30. \quad (4.13)$$

The adjusted probabilities were computed by a single-label softmax:

$$p_{ik} = \frac{\exp(\tilde{z}_{ik})}{\sum_j \exp(\tilde{z}_{ij})}. \quad (4.14)$$

Asymmetric weights were computed from the unsmoothed target. Define

$$x_{\text{pos}} = p \odot q, \quad x_{\text{neg}} = (1 - p) \odot (1 - q), \quad (4.15)$$

$$\gamma = \gamma_{\text{pos}}q + \gamma_{\text{neg}}(1 - q), \quad (4.16)$$

and

$$w = [\max(1 - x_{\text{pos}} - x_{\text{neg}}, 10^{-8})]^\gamma. \quad (4.17)$$

With $\gamma_{\text{pos}} = 0$ and $\gamma_{\text{neg}} = 4$, the true-class term has unit weight, whereas each non-target term is weighted by p_{ik}^4 .

Label smoothing was then applied to the target used in the final loss:

$$\bar{q}_{ik} = (1 - \epsilon)q_{ik} + \frac{\epsilon}{K}, \quad \epsilon = 0.1. \quad (4.18)$$

The per-sample weighted negative log-likelihood and batch reduction were

$$L_i = - \sum_k \bar{q}_{ik} w_{ik} \log p_{ik}, \quad (4.19)$$

$$L_{\text{ASL-LDAM}} = \frac{1}{B} \sum_{i=1}^B L_i. \quad (4.20)$$

The order is therefore margin adjustment, scaling, softmax, asymmetric weighting from the unsmoothed one-hot target, label smoothing, weighted negative log-likelihood, and batch averaging. Figure 4.4 illustrates this training-time computation and its separation from inference.

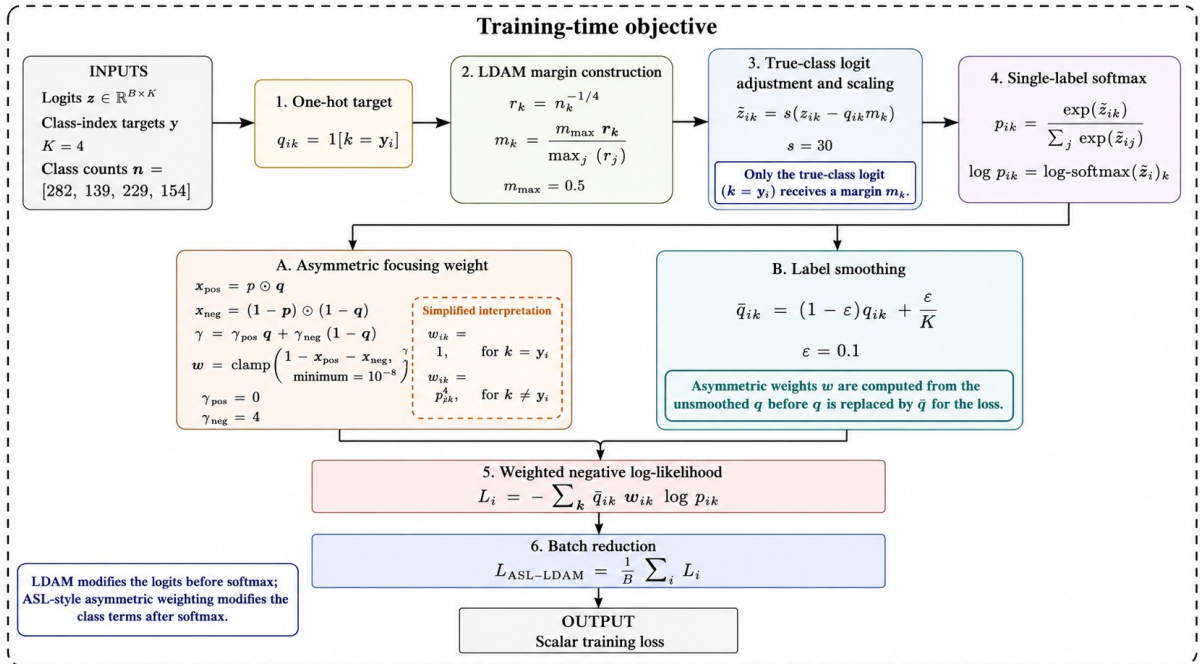


Fig. 4.4. Training-time computation graph of the custom single-label ASL-LDAM objective.

2.6 Classification Training and Evaluation Protocol

The selected classifier was trained with AdamW, an initial learning rate of 0.00125, cosine decay to a terminal learning-rate factor of 0.01, a maximum of 30 epochs, patience of 15 epochs, batch size 32, four data-loading workers, automatic mixed precision, and seed 42. The best checkpoint was selected from validation-side training records and subsequently evaluated under the defined clean, corruption, and additional-source protocols. Table 4.3 records the training configuration.

Table 4.3: Classification training configuration

Parameter	Setting	Parameter	Setting
Reference model	YOLO26m-cls	Input size	224×224
Optimizer	AdamW	Initial learning rate	0.00125
Schedule	Cosine	Final LR factor	0.01
Maximum epochs	30	Patience	15
Batch size	32	Workers	4
Mixed precision	Enabled	Random seed	42
Training augmentation	RandAugment through retained Ultralytics configuration	Checkpoint rule	Validation-selected best checkpoint

Clean evaluation used deterministic preprocessing on the fixed evaluation partition. Controlled synthetic corruption stress testing applied impulse noise, Gaussian noise, contrast reduction, defocus blur, and low-light transformations to the stored background-removed images before tensor conversion. Corrupted variants preserved class labels and file identity. The archived evaluation package organized outputs by corruption family and severity; exact severity parameters remain tied to those retained transformation definitions rather than being inferred from library defaults.

2.6.1 Multi-Source Retraining Protocol

Additional-source classification was evaluated in two retrained regimes: EXT-3-Original and the integrated SDI-4 + EXT-3-Original dataset. EXT-3-Original maps Tom_BT to Healthy, Den_Mang to BG, and Dom_Trang to WSSV; it contains no WSSV_BG category. Each source was partitioned independently using the nominal 70/15/15 classification policy. For the integrated four-class experiment, the corresponding source partitions were merged only after splitting (train + train, validation + validation, and test + test), with WSSV_BG contributed only by SDI-4.

Each regime was retrained and checkpointed from its own validation records; therefore, the experiment is not zero-shot inference with the original four-class checkpoint. Source-wise splitting preserves partition provenance and makes source composition and class-prior changes explicit. Grad-CAM++, HiResCAM, and EigenCAM are retained only as qualitative activation inspections, not as causal, clinical, or mask-accurate evidence.

3. Instance-Segmentation Methodology

3.1 Disease-Region Annotation Protocol and Label Semantics

The official ShrimpDiseaseImageBD source provides image-level disease categories but does not provide instance-segmentation polygons or masks. The project team therefore created the disease-region

annotations used in this branch. The segmentation label space contained two positive mask classes: BG and WSSV. WSSV_BG remained an image-level classification category and was not introduced as a third mask class. In co-infection images, spatially separable BG and WSSV evidence could be annotated as separate instances.

BG polygons were assigned to visibly darkened gill regions, whereas WSSV polygons were assigned to visible white-spot or white-lesion regions. Shadows, reflections, background marks, and weak texture changes were excluded when they were not visually consistent with the target symptom. Healthy was never defined as a segmentation class and received no positive disease-mask supervision.

All masks were produced by the project team rather than by an aquaculture pathologist. No independent expert boundary review or inter-annotator agreement study was available. The masks therefore represent a consistent project annotation protocol, not clinical ground truth, and this is a material limitation of the study.

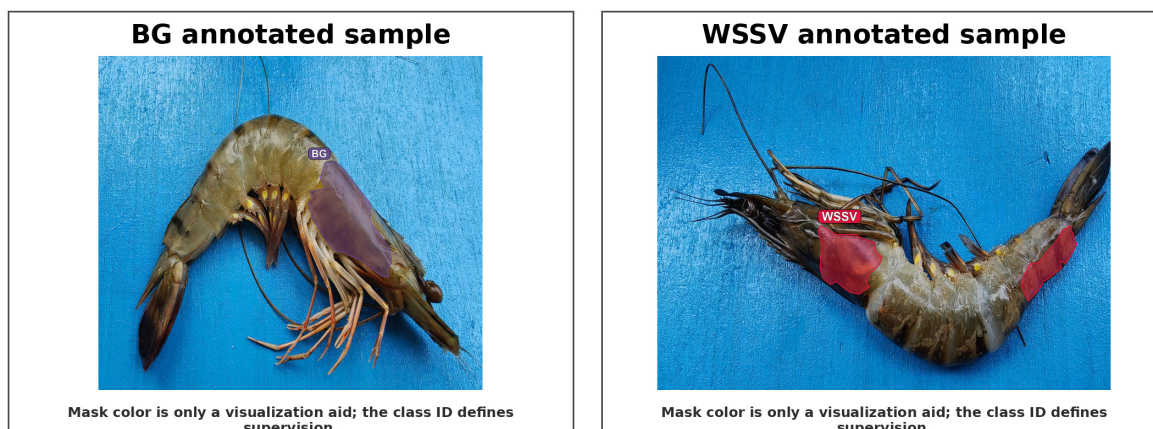


Fig. 4.5. Representative project-authored BG and WSSV disease-region annotations derived from ShrimpDiseaseImageBD. Polygon colors are visualization aids only; training uses the stored class identifiers and polygon coordinates.

Figure 4.5 illustrates the annotation semantics used by the conversion scripts and dataset manifests. Overlay colors are visualization aids only; the stored class identifier and polygon coordinates define the training target.

Table 4.4: Segmentation annotation rules and label semantics

Label/status	Positive annotation rule	Exclusion or handling rule
BG	Visibly darkened gill tissue with a coherent disease-region boundary	Exclude general shadows, background darkness, and uncertain color changes.
WSSV	Visible white spots or white-lesion regions consistent with WSSV appearance	Exclude glare, isolated reflections, and background white marks.
Co-infection image	Annotate separable BG and WSSV regions as their respective classes	If project-authored polygons overlap, retain one deterministic target; WSSV has project-specific priority in overlapping pixels.
Healthy	No positive disease-mask class; empty label file	Retained in train/validation/test as a known negative sample for negative supervision and Healthy false-positive diagnostics.
Ambiguous region	None	Omit uncertain pixels rather than forcing a disease label.
Display color	Visualization only	Class identity is determined by the stored class ID, not by overlay color.

Ultralytics YOLO instance segmentation supports multiple instances and multiple classes in the same image. The project therefore retained spatially separable BG and WSSV regions as independent class-specific instances in co-infection images. A multilabel semantic target was not assigned to the same annotated pixel. When project-authored BG and WSSV polygons overlapped, the overlap was resolved by one deterministic project rule in which WSSV took priority. This rule keeps stored targets consistent; it is neither a limitation of YOLO’s ability to represent multiple disease instances nor a biological claim that one disease replaces the other.

Healthy images are known negative samples rather than missing annotations. Their label files contain no positive disease polygon because no visible BG or WSSV region is present. Consequently, no matched positive instance contributes box or mask regression terms on those images, while the detector is trained against spurious disease-instance predictions. Healthy behavior is then evaluated separately through Healthy false-positive diagnostics.

3.2 Leakage-Aware Specimen-Grouped Partitioning

Repeated photographs of the same physical shrimp create a train-test leakage risk. For the convention-matched portion of the segmentation source, filenames were parsed using the pattern Disease-ShrimpID-img-ImageNumber.jpg. Because numeric identifiers can repeat across disease folders, the grouping key combines the disease prefix and shrimp identifier.

For convention-matched filenames, the specimen key is defined as

$$G_i = \text{Disease}_i :: \text{ShrimpID}_i. \quad (4.21)$$

A valid grouped partition satisfies

$$G_i = G_j, \quad S_i = S_j. \quad (4.22)$$

Every known image view sharing one specimen key is therefore assigned to one partition. The principal segmentation experiments use disease-stratified specimen groups with a nominal 80/10/10 train/validation/test target. Because specimen groups are indivisible, the realized image counts need not equal the nominal percentages exactly. Leakage is reported at specimen-group level rather than as duplicate-image overlap.

3.2.1 Original and Expanded Instance-Segmentation Dataset Construction

The convention-matched segmentation subset contains 1,149 images associated with 416 filename-derived specimen groups. Of these images, 746 contain at least one project-authored BG or WSSV mask, while 403 Healthy images are retained as empty-label negatives. The subset contains 1,031 annotated disease instances (462 BG and 569 WSSV). WSSV_BG remains an image-level category; co-infection images use BG and WSSV as the two instance-mask classes rather than a third co-infection mask class.

Table 4.4A: Recognized specimen-group subset under the nominal 80/10/10 grouped protocol

Split	Images	Specimens	Labeled disease images	Healthy empty-label images
Train	905	331	584	321
Validation	115	40	74	41
Test	129	45	88	41
Total	1,149	416	746	403

The expanded inventory adds 303 BG/WSSV-positive images whose filenames do not encode a compatible specimen identifier. Consequently, those images cannot be assigned by specimen group from filename evidence. The 1,149 convention-matched images preserve grouped allocation, whereas the 303 unmatched-name images are assigned by image-level stratified splitting before the partitions are merged. The resulting 1,147/144/161 train/validation/test inventory must therefore not be described as fully specimen-leakage-free.

Table 4.4B: Expanded instance-segmentation image inventory

Split	Images	Convention-matched images	Added unmatched images
Train	1,147	905	242
Validation	144	115	29
Test	161	129	32
Total	1,452	1,149	303

Figure 4.6 illustrates the split-policy logic only for convention-matched specimen keys. It must not be interpreted as evidence that the 303 unmatched-name images were specimen-grouped.

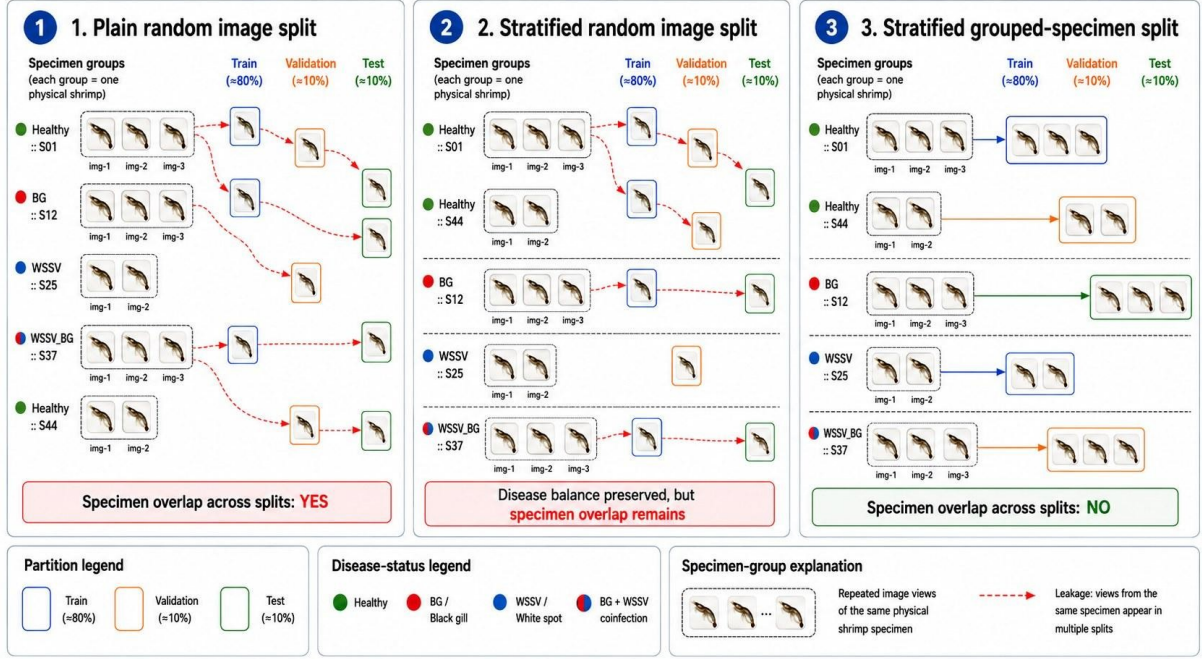


Fig. 4.6. Comparison of plain random, stratified random, and stratified grouped-specimen partitioning, with the grouped policy preventing repeated-specimen overlap across partitions.

The definitions and uses of the three policies are summarized in Table 4.5. Split manifests, specimen keys, class summaries, and file fingerprints were preserved so that partition integrity could be checked independently of training code.

Table 4.5: Segmentation split-policy definitions

Policy	Assignment unit and constraint	Methodological use
Plain random image split	Individual images assigned without disease stratification or specimen grouping	Leakage-prone comparison condition
Stratified random image split	Individual images assigned while approximating disease proportions	Controls class balance but not repeated-specimen overlap
Stratified grouped-specimen split	Specimen groups assigned as indivisible units while approximating disease proportions	Principal segmentation training, validation, and evaluation protocol

Table 4.5 separates class-balance control from identity-leakage control. Quantitative differences among these policies are reported in Chapter VI rather than here.

3.3 YOLO11n-seg Reference Architecture and Augmentation Controls

YOLO11n-seg was retained after the grouped-specimen baseline screen using a multi-objective rule combining localization, parameter efficiency, Healthy-negative behavior, and HScore rather than a single AP metric. The choice does not imply dominance of every metric. Images were processed at 640×640 ; immediately before the Segment head, the neck produced the P_3 , P_4 , and P_5 tensors shown in Equation (4.23).

$$P_3 : B \times 64 \times 80 \times 80, \quad P_4 : B \times 128 \times 40 \times 40, \quad P_5 : B \times 256 \times 20 \times 20. \quad (4.23)$$

The reference architecture was intentionally treated as a stable segmentation scaffold rather than as a target for wholesale redesign. This distinction matters because the attention experiments were intended to test feature recalibration around an otherwise recognizable YOLO11n-seg path. Retaining the original backbone, PAN-FPN neck, and Segment head keeps the output interface, mask semantics, and training target structure fixed while allowing the intervention to be localized to the feature tensors immediately before the head. In practical terms, the comparison asks whether the information delivered to the existing head can be made more useful without changing what the head is asked to predict.

The three feature levels provide complementary spatial roles. P3 has the highest spatial resolution and therefore carries relatively fine edge, texture, and small-region evidence. P4 combines a larger receptive field with still-meaningful spatial detail, making it suitable for intermediate lesion structure and boundary patterns. P5 is spatially coarser but carries broader semantic context. Applying a shape-preserving recalibration at all three levels allows the later Segment head to receive modified multi-scale evidence without requiring a new decoder, new class definition, or different mask representation. The design therefore isolates feature selection from output semantics more cleanly than replacing the complete segmentation head.

The grouped-specimen protocol is also part of the architectural selection logic. A segmentation model that appears strong only when repeated views of the same specimen cross the split boundary would be a poor methodological reference for subsequent attention experiments. The retained reference was therefore chosen from the grouped evaluation regime and interpreted jointly with localization quality, parameter count, Healthy-negative behavior, and the project-defined HScore. This multi-objective rule is important because a model can improve disease-region AP while simultaneously producing an undesirable number of false disease masks on Healthy images. The reference decision consequently emphasizes a usable balance rather than a single maximum metric.

The augmentation controls were separated for the same reason. “Clean-light” provides moderate perturbation while preserving a comparatively restrained training distribution. “Strict control” removes the explicit online perturbations and the hidden Albumentations path so that image-space Fourier experiments can isolate deterministic preprocessing. The “stronger candidate” increases photometric and geometric variability for selected attention runs while leaving mosaic, MixUp, CutMix, copy-paste, and the hidden Albumentations hook disabled. These policies are not interchangeable labels: they define different experimental conditions and are therefore retained explicitly in the run ledger instead of being collapsed into a generic statement that augmentation was used.

Finally, the reference architecture does not reinterpret Healthy as a third mask class. Healthy images remain empty-label negatives, whereas BG and WSSV are the only positive instance-mask classes. This is essential for interpreting later false-positive diagnostics: a Healthy error is the appearance of one or more retained disease predictions on an image that contains no positive disease polygon, not a failure to predict a “Healthy mask.” The architecture, label semantics, grouped split, and augmentation policy therefore form one methodological unit. Chapter VI reports the resulting trade-offs; this section defines the conditions under which those comparisons are meaningful.

P3 retains fine spatial detail, P4 represents intermediate disease and boundary patterns, and P5 provides broader semantic context. Added attention modules preserve these tensor shapes and return three tensors to the unchanged Segment head.

Three segmentation augmentation policies are distinguished so that preprocessing differences remain explicit. Clean-light applies restrained photometric and geometric perturbations; strict control disables all explicit online augmentation and the hidden Albumentations hook; the stronger candidate increases color and geometric variation while keeping mosaic, MixUp, CutMix, and copy-paste disabled.

Table 4.6: Segmentation augmentation and control policies

Policy	Photometric settings	Geometric settings	Disabled / inactive components
Clean-light	HSV h/s/v = 0.01 / 0.35 / 0.20	Horizontal flip 0.50; translation 0.05; scale 0.20	Vertical flip, rotation, shear, perspective; mosaic, MixUp, CutMix, copy-paste; auto augmentation
Strict control	All explicit photometric augmentation set to zero	All explicit geometric augmentation set to zero	Hidden Albumentations hook and all online augmentation
Stronger candidate	HSV h/s/v = 0.05 / 0.50 / 0.40	Horizontal/vertical flip 0.50/0.30; rotation 10 deg; translation 0.10; scale 0.50	Mosaic, MixUp, CutMix, copy-paste, hidden Albumentations hook; erasing = 0.15 recorded but inactive

Random erasing is not treated as an executed component of the stronger segmentation policy. Although erasing = 0.15 appears in the recorded arguments, the retained Ultralytics segmentation transform path did not apply it. Table 4.6 therefore distinguishes configuration fields from transformations that actually affected the images. Broader attention runs also differ in augmentation policy, epoch budget, early stopping, initialization, framework version, placement, and auxiliary loss; those run-specific factors are preserved in Table 4.7.

3.4 Attention-Based Multi-Scale Feature Recalibration

3.4.1 Coordinate Attention Followed by SimAM

Coordinate Attention (CA) first summarizes each feature map along the two spatial axes. For channel c , the directional descriptors are

$$z_c^h(h) = \frac{1}{W} \sum_{i=1}^W x_c(h, i), \quad z_c^w(w) = \frac{1}{H} \sum_{j=1}^H x_c(j, w). \quad (4.24)$$

After shared transformation and direction-specific projection, the coordinate gates recalibrate the input as

$$y_c(i, j) = x_c(i, j) g_c^h(i) g_c^w(j). \quad (4.25)$$

SimAM is then applied to the CA output at P3, P4, and P5 before the unchanged Segment head. The placement and subsequent attention-family protocol are detailed on the following pages.

SimAM was then applied using the energy formulation in the equations above. The order was CA followed by SimAM at P_3 , P_4 , and P_5 . The outputs preserved the input shapes and were passed to the original Segment head. Figure 4.7 shows the insertion points.

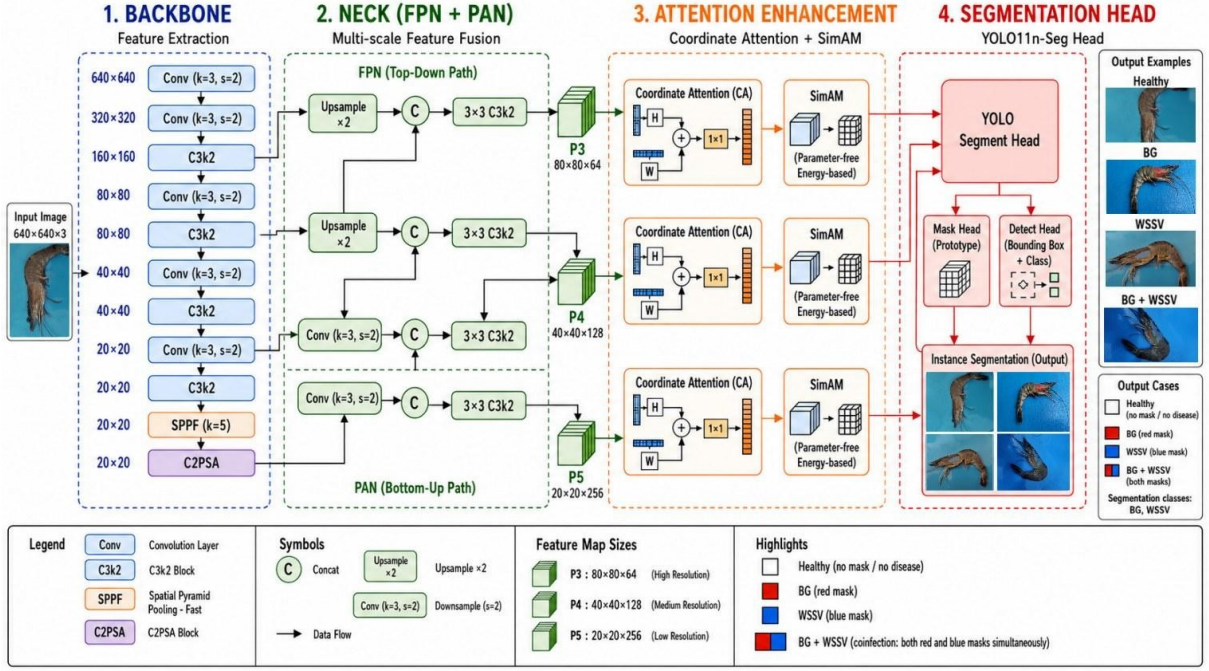


Fig. 4.7. YOLO11n-seg architecture with Coordinate Attention followed by SimAM at the P3, P4, and P5 feature-pyramid outputs before the original Segment head.

3.4.2 LKA-to-SimAM

A second multi-scale configuration placed a large-kernel attention operator before SimAM at P_3 , P_4 , and P_5 . The large-kernel stage was intended to aggregate wider spatial context before the parameter-free energy recalibration. All three tensors retained their original channel counts and spatial dimensions, and the backbone, PAN-FPN neck, and Segment head were unchanged. This configuration was treated as a separate controlled run rather than evidence of superiority, particularly because its augmentation and stopping protocol differed from the CA-to-SimAM run.

3.4.3 Dual-Polarity Contrast Attention

Dual-Polarity Contrast Attention (DPCA) was a project-created, shape-preserving feature-recalibration module. For input $x \in \mathbb{R}^{B \times C \times H \times W}$, a 5×5 average pool defined the local context:

$$c = \text{AvgPool}_{5 \times 5}(x), \quad d = x - c. \quad (4.26)$$

Positive and negative deviations were separated as

$$b = \text{ReLU}(d), \quad q = \text{ReLU}(-d). \quad (4.27)$$

A grouped 1×1 convolution produced a channel-position selector:

$$w = \sigma \left(\text{Conv}_{1 \times 1}^{\text{grouped}}([b; q]) \right). \quad (4.28)$$

The selected evidence and depthwise gate were

$$e = w \odot b + (1 - w) \odot q, \quad (4.29)$$

$$g = \sigma(\text{DWConv}_{3 \times 3}(e)). \quad (4.30)$$

The residual output was

$$y = x + \gamma x \odot (g - 0.5), \quad \gamma_0 = 0. \quad (4.31)$$

The grouped selector used bias, the depthwise refinement convolution did not use bias, and the module was applied at P_3 , P_4 , and P_5 . Initializing γ to zero made the starting mapping identical to the reference feature path. Figure 4.8 summarizes the implementation.

The LKA-to-SimAM configuration was included to test a different ordering of spatial context aggregation and parameter-free energy recalibration. The large-kernel stage is able to integrate information over a wider effective neighborhood before SimAM evaluates the relative salience of individual activations. Conceptually, this sequence asks whether broader contextual filtering can produce a more informative tensor for the subsequent energy-based weighting step. Because the operator is inserted at P3, P4, and P5 without changing tensor dimensions, the original Segment head receives the same number of feature maps with the same channel and spatial sizes as in the reference model.

That architectural similarity should not be mistaken for a strict causal ablation. The retained LKA-to-SimAM run differs from the CA-to-SimAM run in augmentation and stopping conditions, as preserved in Table 4.7. A difference in final localization therefore reflects the complete recorded configuration, not an isolated estimate of “LKA versus CA.” The methodological role of this branch is narrower: it provides an alternative multi-scale recalibration strategy under its actual training record and helps test whether the selected CA-to-SimAM configuration is merely one arbitrary attention insertion among several plausible choices.

DPCA was designed around a different hypothesis. Instead of first summarizing features across long spatial axes or a large kernel, it separates each activation from a local average context. Positive deviations represent responses brighter or stronger than their local neighborhood, while negative deviations represent the opposite polarity. The grouped selector then learns how much of each polarity should contribute at each channel-position location. A depthwise gate refines that selected evidence, and the residual form returns the result without changing the feature shape.

The zero initialization of the residual coefficient is a deliberate stability control. At initialization, the DPCA branch contributes no perturbation and the module behaves like the identity mapping. Training can then increase or decrease the residual contribution from that neutral starting point. This avoids forcing a randomly initialized attention transformation into all three pyramid levels before the rest of the pretrained network has adapted. It also makes the module’s starting behavior easier to audit because the initial forward path is identical to the reference feature path apart from numerically negligible implementation effects.

The dual-polarity construction is an engineering representation of local contrast, not a biological model of shrimp pathology. A positive or negative feature deviation cannot be interpreted directly as a specific disease symptom. Its purpose is to give the network a structured way to retain both increases and decreases relative to local context, which may be relevant to darkened gill regions, bright lesion-like responses, boundaries, or unrelated texture. Any biological interpretation must therefore come from validated downstream evidence rather than from the algebra of the attention module itself.

Across CA-to-SimAM, LKA-to-SimAM, and DPCA, the common methodological constraint is that the backbone-to-head interface remains shape compatible. This permits the later comparison to focus on feature recalibration choices while keeping the segmentation classes, head outputs, dataset semantics, and evaluation definitions fixed. Differences in training protocol are disclosed rather than erased, and the broader family is therefore treated as configuration-level evidence. The more narrowly controlled placement experiment in Section 3.5 addresses insertion location separately.

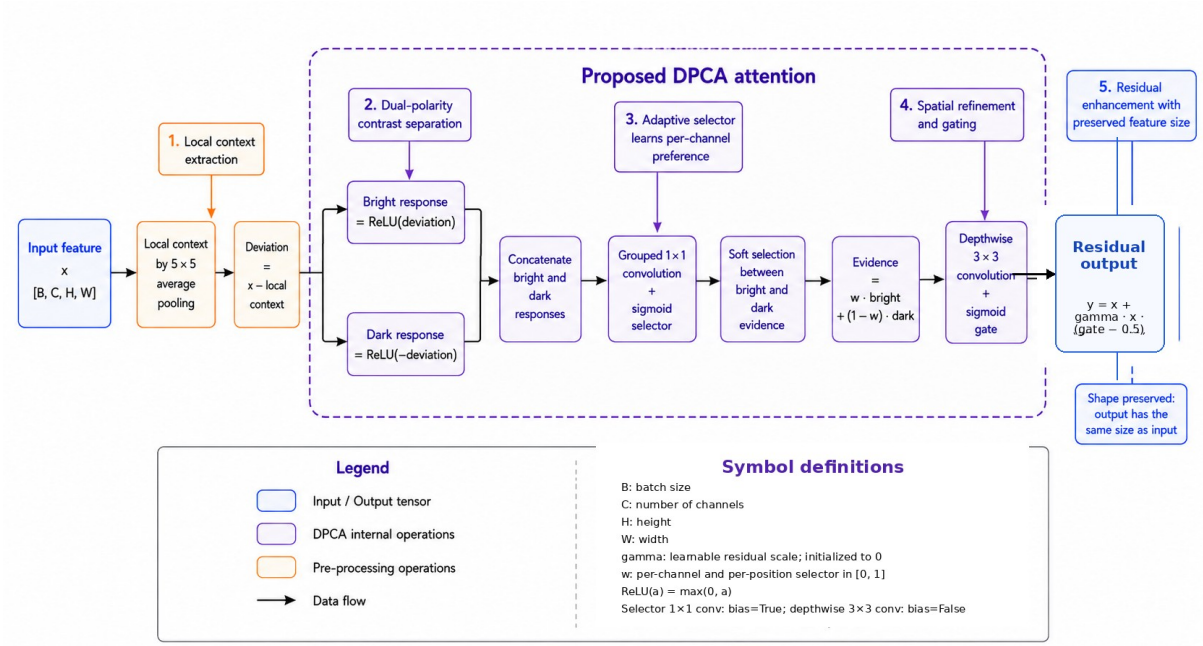


Fig. 4.8. Module-level workflow of Dual-Polarity Contrast Attention, including local context extraction, polarity separation, grouped selection, depthwise gating, and zero-initialized residual scaling.

3.4.4 CoTEGate and BoundaryLite

CoTEGate was a project-created residual feature-recalibration module applied only to P_4 ; P_3 and P_5 bypassed it. A TripletLiteGate generated a spatial response T from H-W, C-W, and H-C views, while an Efficient Channel Attention path generated channel response C . Their disagreement was

$$D = \text{AvgPool}_{3 \times 3}(|T - C|). \quad (4.32)$$

The consensus gate and residual output were

$$G = \sigma(\lambda_t T + \lambda_c C - \alpha D), \quad (4.33)$$

$$y = x + \gamma x \odot G. \quad (4.34)$$

The initial values were $\lambda_t = 1.0$, $\lambda_c = 1.0$, $\alpha = 0.20$, and $\gamma = 0.0$. The zero-initialized residual scale preserved the reference mapping at initialization.

BoundaryLite was not an attention module. It was a training-only auxiliary boundary loss. For a binary ground-truth mask M , a soft boundary target was constructed by local max/min pooling:

$$B(M) = \text{MaxPool}(M) - \text{MinPool}(M). \quad (4.35)$$

Boundary binary cross-entropy was evaluated only for positive mask instances and added to the standard mask BCE:

$$L_{\text{mask}} = L_{\text{BCE}} + \lambda_b L_{\text{boundary}}, \quad \lambda_b = 0.03. \quad (4.36)$$

Because this configuration changed both the P_4 feature path and the training objective, it was not interpreted as an attention-only ablation. Figure 4.9 shows the architectural and loss-side components separately.

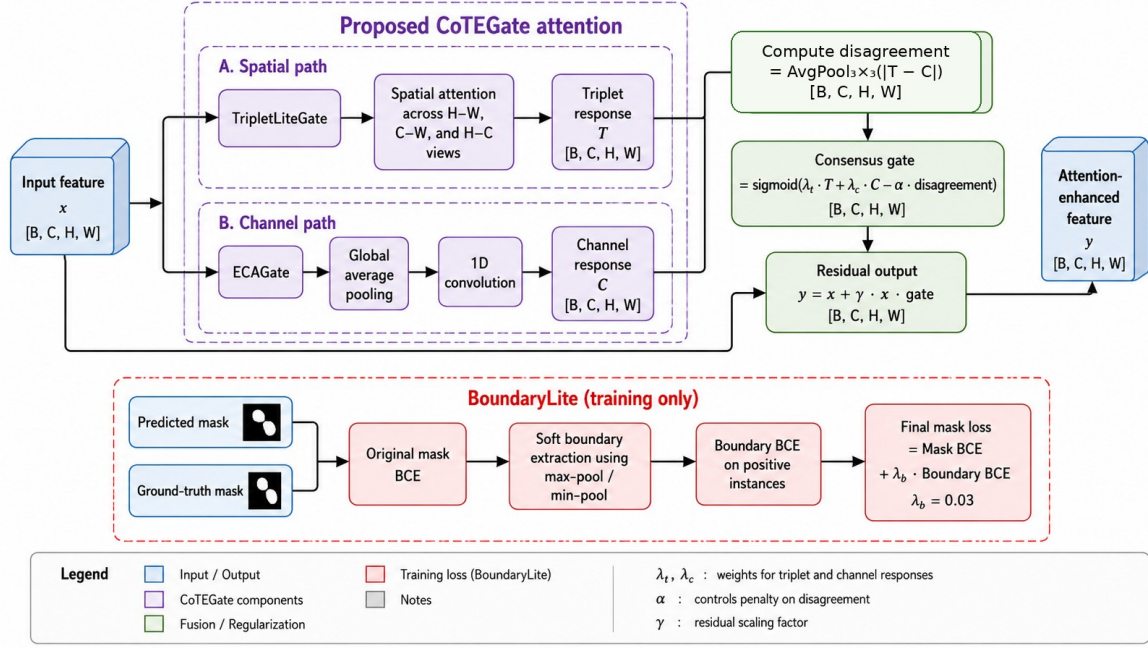


Fig. 4.9. CoTEGate residual recalibration at P4 and the training-only BoundaryLite auxiliary mask-boundary loss.

The recorded attention protocols are compared in Table 4.7. The table reports implementation conditions, not predictive outcomes.

Table 4.7: Recorded attention configuration protocols

Configuration	Placement	Aug. policy	Epochs	Initialization	Ultralytics / PyTorch
YOLO11n-seg reference	None	Clean-light	100 / 60, early stop	yolo11n-seg.pt	8.4.61 / 2.10.0+cu128
CA-to-SimAM	P3/P4/P5	Stronger	100 / 100	yolo11n-seg.pt; 594/594 tensors	8.4.67 / 2.11.0+cu128
LKA-to-SimAM	P3/P4/P5	Clean-light	100 / 75, early stop	yolo11n-seg.pt; 579/579 tensors	8.4.61 / 2.10.0+cu128
DPCA	P3/P4/P5	Stronger	200 / 200	yolo11n-seg.pt	8.4.102 / 2.10.0+cu128
CoTEGate + BoundaryLite	P4 + auxiliary loss	Stronger	200 / 184, early stop	yolo11n-seg.pt	8.4.95 / 2.10.0+cu128

Table 4.7 preserves the run-specific protocol differences that materially affect interpretation. Augmentation policy, epoch budget, early stopping, initialization, framework version, placement, and the BoundaryLite auxiliary objective were not identical across the broader attention family. Accordingly, these rows define configuration-level experiments and are not treated as a strict one-factor causal ablation.

3.5 Attention Efficiency and Placement-Control Protocol

Implementation efficiency was recorded alongside the architectural protocol so that the later localization results could be interpreted jointly with model complexity. The parameter counts below are exact retained model counts. The millisecond and FPS values were measured in the experimental notebook GPU environment and are not Android runtime measurements.

Table 4.8: Attention implementation and efficiency metadata

Configuration	Placement / objective	Parameters	Inf. ms/image	FPS
YOLO11n-seg baseline	No added attention	2,842,998	6.0	166.7
CA → SimAM	P3/P4/P5	2,854,718	9.3	107.5
LKA → SimAM	P3/P4/P5	2,963,510	8.6	116.3
DPCA	P3/P4/P5	2,848,377	8.8	113.6
CoTEGate + BoundaryLite	P4 + auxiliary loss	2,736,562	8.2	122.0

A separate CA → SimAM placement-control study moved the same recalibration operator across seven retained insertion patterns: P3/P4/P5, neck concatenation, head P3, head P4, after C2PSA, backbone P3, and neck P3/P4. The grouped dataset and evaluation definitions were retained while the insertion site was varied within that placement study. This experiment supports the placement decision reported in Chapter VI; it does not retroactively make the broader attention-family table a strict ablation.

4. Fourier-Domain Segmentation Methodology

4.1 Frequency-Domain Formulation

For an image channel $x(m, n)$ of size $M \times N$, the two-dimensional discrete Fourier transform was defined as

$$X(u, v) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x(m, n) \exp \left[-j2\pi \left(\frac{um}{M} + \frac{vn}{N} \right) \right]. \quad (4.37)$$

A radial filter $H(u, v)$ modified selected frequency components:

$$\tilde{X}(u, v) = H(u, v)X(u, v), \quad (4.38)$$

and the transformed channel was reconstructed by inverse DFT:

$$\tilde{x}(m, n) = \text{Re}\{IDFT2(\tilde{X})\}. \quad (4.39)$$

Image-space Fourier processing changes pixel appearance without changing spatial coordinates. Polygon vertices and mask geometry therefore remain aligned unless a separate geometric augmentation is applied.

4.2 Image-Level Fourier Candidate Families

The candidate screen covered high-pass enhancement, high-frequency damping, restricted radial bands, and broad low-frequency flattening. These operators define the search space used before W1 was fixed as the reference transform.

The image-level Fourier screen was organized as a set of deliberately different frequency manipulations rather than as many numerical variants of one filter. High-pass enhancement preserves the original image and adds a scaled residual derived from content outside a Gaussian low-frequency component. High-frequency damping moves in the opposite direction by suppressing selected fine-scale content. Band-pass processing emphasizes or retains a restricted radial range, while low-frequency flattening reduces broad image components associated with slowly varying illumination and background structure. Taken together, these families span qualitatively different ways of redistributing image energy before segmentation inference.

The recorded parameter sweep in Table 4.9 therefore has two roles. First, it documents the exact candidate settings that were available to the retained experiment. Second, it prevents the methodology from being summarized after the fact as if only the eventually retained W1 configuration had been considered. The table is a search-space ledger, not a result ranking. Candidate performance is intentionally deferred to Chapter VI so that the present chapter describes what was changed in the input and how the changes were controlled, without mixing procedure with outcome.

All image-space candidates operate channel-wise on RGB values and preserve the image lattice. They do not translate, rotate, crop, or rescale the object geometry by themselves. Consequently, an existing polygon remains spatially aligned after a pure Fourier transform, subject only to the photometric change in pixel values. This property is methodologically useful because it separates frequency manipulation from label-coordinate transformation. When a geometric augmentation is also used, polygon coordinates are updated for that geometric operation; the Fourier component does not require an additional coordinate mapping.

The diagnostic visualizations use several complementary views because a transformed RGB image alone can hide the actual operation. The spatial absolute-difference panel shows where pixel values changed. Log-power spectra provide a compact view of frequency redistribution, while a signed spectrum difference indicates whether energy increased or decreased relative to the matched reference. Radial summaries reduce the two-dimensional spectrum to a frequency-distance profile, making it easier to inspect whether a candidate behaves consistently with its intended low-, mid-, or high-frequency role. These views are checks on implementation and image distribution, not substitutes for segmentation metrics.

The candidate families also differ in how strongly they can interact with clipping and quantization. Any operator that increases local contrast may push reconstructed values outside the valid image range and therefore require clipping before uint8 storage or later preprocessing. Once clipping occurs, the complete stored-image transformation is no longer represented by a single linear frequency response. For this reason, the methodology distinguishes the theoretical filter operation from the final bounded image that is actually consumed by the model.

No candidate is assumed to be beneficial simply because it increases high-frequency content or makes edges appear sharper. Fine-scale energy can correspond to useful lesion boundaries, sensor noise, compression artifacts, background texture, or augmentation-induced structure. The screen is therefore empirical and controlled: operators are defined mathematically, candidate settings are preserved, geometry is held fixed unless explicitly augmented, and predictive interpretation is postponed to the evaluation chapter. This separation is what allows W1 to be treated later as a reproducible reference configuration rather than as an intuitively chosen image enhancement.

Table 4.9: Fourier candidate families and recorded sweep settings

Candidate	Recorded settings	Intended frequency operation
High-pass enhancement	sigma = 35, 40, 45, 50, 55; alpha = 0.05, 0.08, 0.10, 0.12, 0.15, 0.20, 0.30	Add a scaled high-frequency residual to the image
High-frequency damping	sigma = 50 or 70; alpha = 0.05 or 0.10	Suppress selected high-frequency detail
Band-pass filtering	radii 12/60 and 20/80; alpha = 0.20	Retain or boost a restricted radial band
Low-frequency flattening	sigma = 100; beta = 0.30	Reduce broad illumination/background components

Candidate visualizations compare the original image and mask, transformed image, spatial difference, log-power spectra, and spectrum change. Predictive rankings remain in Chapter VI; this section defines only the operators and controls used to generate them.

4.3 W1 High-Pass Reference Configuration

W1 was fixed as the image-space high-pass reference for order, corruption, and spectrum-interaction experiments. For each image channel x_c ,

$$F_c = FFT2(x_c). \quad (4.40)$$

A Gaussian low-pass mask was

$$M_\sigma(u, v) = \exp \left[-\frac{u^2 + v^2}{2\sigma^2} \right]. \quad (4.41)$$

The low-frequency reconstruction and enhanced channel were

$$x_{low,c} = IFFT2(M_\sigma F_c), \quad (4.42)$$

$$x'_c = clip[x_c + \alpha(x_c - x_{low,c}), 0, 255]. \quad (4.43)$$

The retained W1 setting uses sigma = 50 and alpha = 0.10. The transform is applied channel-wise, retains the full image, and adds only a scaled high-frequency residual before clipping to the valid image range. Because the operation does not move pixels, it preserves image dimensions and polygon coordinates.

The W1 definition can be read as a residual formulation rather than as a replacement filter. The Gaussian mask first reconstructs a smooth low-frequency component, and subtracting that component from the original produces a high-frequency residual. Multiplying the residual by α controls how strongly that detail is reintroduced, while σ controls the spatial-frequency scale represented by the Gaussian low-pass term. With the retained setting $\sigma = 50$ and $\alpha = 0.10$, the operation is intentionally modest: the original image remains the dominant component and the added residual acts as a limited emphasis rather than a complete high-pass reconstruction.

This residual form is important for both interpretability and label preservation. Because the original image is retained, large-scale appearance and object location are not discarded. Because the operation is pointwise after inverse transformation and does not resample the image grid, mask vertices remain valid in the same coordinate system. The transform can change the local contrast of a lesion boundary or background texture, but it cannot by itself move a polygon to another location. That distinction is why W1 can be composed with separate geometric augmentation without requiring a second mask-coordinate transform for the Fourier step.

Before clipping, the operation has a direct linear interpretation. Frequencies that lie close to the Gaussian low-pass center receive little additional residual gain, whereas frequencies farther from the center approach a multiplicative response near $1 + \alpha$. The exact response depends on the Gaussian mask, but the qualitative behavior is fixed by the equations rather than inferred from a displayed image. After inverse transformation, however, pixel values are clipped to the allowed range and may be quantized for storage. Those post-processing steps are nonlinear, so the final stored RGB image should not be described as if one ideal linear transfer function explained every pixel.

W1 was fixed as the reference transform so that later experiments could vary one surrounding factor without repeatedly redefining the Fourier operator. The order study changes whether augmentation occurs before or after W1. The paired spectrum protocol holds W1 constant while comparing matched stochastic augmentation paths. The corruption protocol applies synthetic degradation before the retained Fourier step. Using one fixed σ and α setting across these branches makes the downstream comparisons traceable and prevents a different filter from being silently selected for each experiment.

The reference is deterministic for a given input image and implementation. It contains no learned weights, has no dependence on the segmentation checkpoint, and does not adapt its parameters to the predicted class or mask. That makes it methodologically distinct from the feature-domain Fourier branch in Section 4.5, where frequency manipulation occurs inside the network and introduces trainable parameters. Keeping image-space W1 and feature-space refinement separate avoids attributing a network-architecture effect to an input preprocessing operation.

Figures 4.10 and 4.11 therefore serve as implementation audits rather than performance evidence. The first view summarizes the visible and spectral effect of the retained setting, while the decomposition view exposes the low-frequency reconstruction, residual, scaled residual, clipping stage, and corresponding spectra. Together they verify that the operation implemented in later experiments matches the stated equations and that the limits of the linear interpretation are explicit before any segmentation result is discussed.

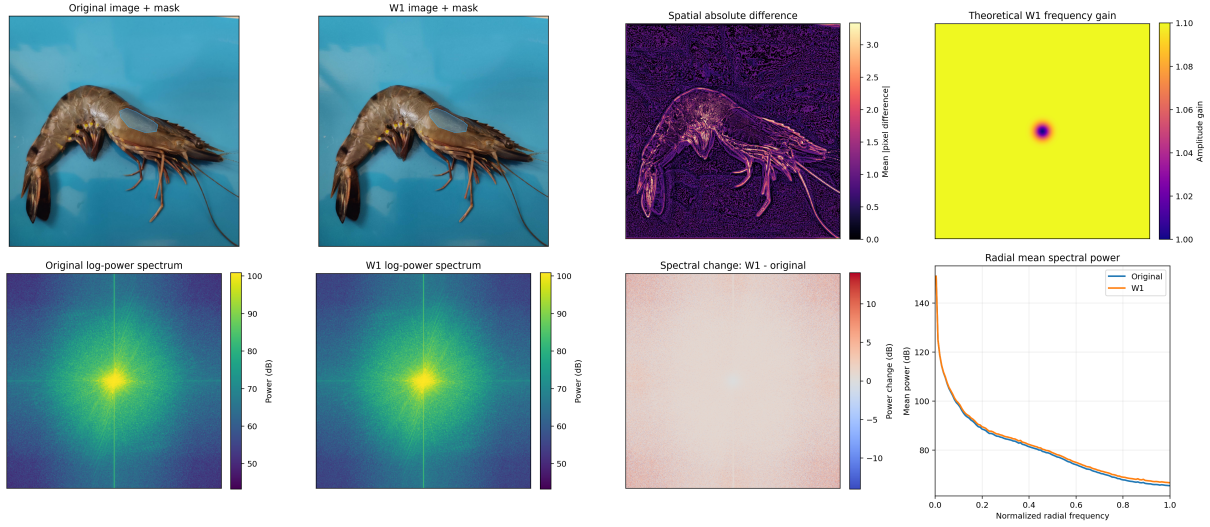


Fig. 4.10. Representative spatial- and frequency-domain effect of W1 (sigma = 50, alpha = 0.10). The panels compare the original and transformed image, spatial absolute difference, theoretical frequency gain, original and W1 log-power spectra on a shared scale, signed spectral change, and radial mean spectral power. The original remains recognizable because the full image is retained and only a scaled high-frequency residual is added.

Figure 4.10 is intended to make the operator visually interpretable rather than to serve as a performance result. The original image remains recognizable because W1 does not replace low-frequency content; it adds a low-amplitude residual whose gain approaches $1 + \alpha$ away from the Gaussian low-pass center. The spatial difference emphasizes edges and fine texture, while the radial-power profile verifies that the transformed image receives a modest frequency-dependent emphasis rather than an arbitrary geometric change.

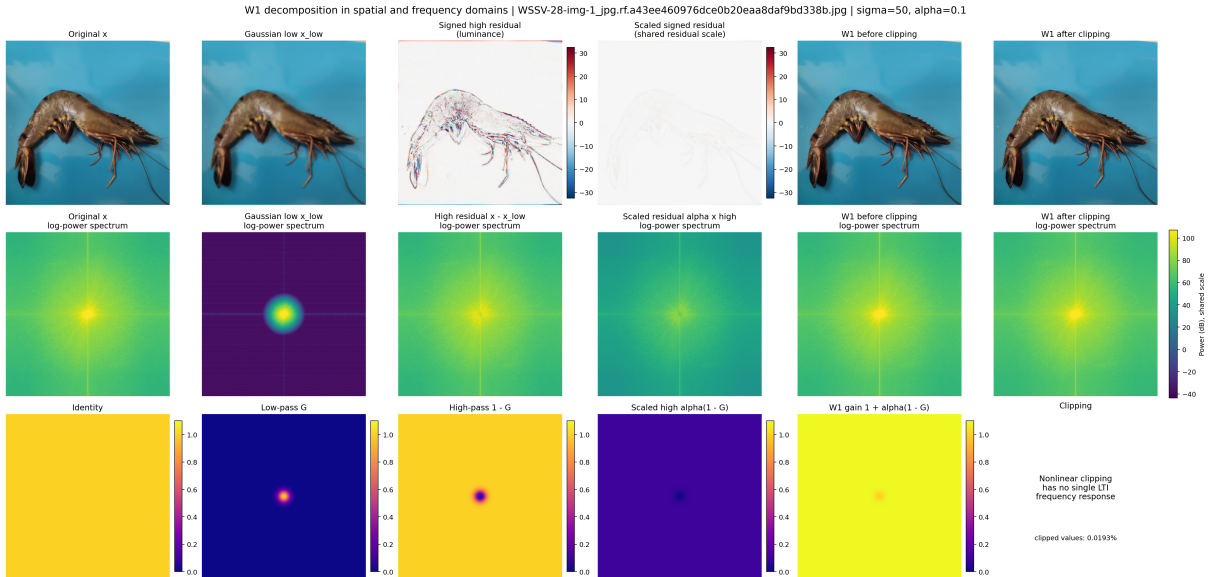


Fig. 4.11. Exact W1 decomposition in the spatial and frequency domains. The figure shows the original image, Gaussian low-frequency reconstruction, signed high-frequency residual, scaled residual, W1 result before clipping, W1 result after clipping and uint8 quantization, their log-power spectra, and the corresponding theoretical filter responses.

The decomposition audit verifies $x = x_{low} + x_{high}$ up to floating-point precision before clipping. This distinction is important because the FFT-domain derivation is linear only through residual addition. Clipping and integer quantization are nonlinear post-processing operations and therefore do not have one exact linear shift-invariant frequency response. Any interpretation of W1 as a simple frequency gain

applies to the pre-clipping linear stage, while the final stored image also reflects those bounded non-linearities.

4.4 Paired Image-Spectrum Interaction Protocol

To examine how W1 interacts with the retained strong segmentation augmentation policy, a paired image-spectrum study was conducted on 24 held-out seed-42 test images: six Healthy, six BG, six WSSV, and six WSSV_BG. Each image was processed under 12 matched stochastic augmentation realizations and five paths: original, W1 only, strong augmentation only, W1 followed by strong augmentation, and strong augmentation followed by W1. The same sampled augmentation parameters were reused across matched paths for each image-realization pair.

The active strong-policy replica followed the recorded Ultralytics 8.4.62 segmentation transform order for letterbox resizing, rotation, translation, scale, HSV transformation, vertical flip, and horizontal flip. Mosaic, MixUp, CutMix, copy-paste, shear, perspective, and the hidden Albumentations hook were disabled. The recorded erasing=0.15 argument is not treated as active because it was ignored by the segmentation v8_transforms path.

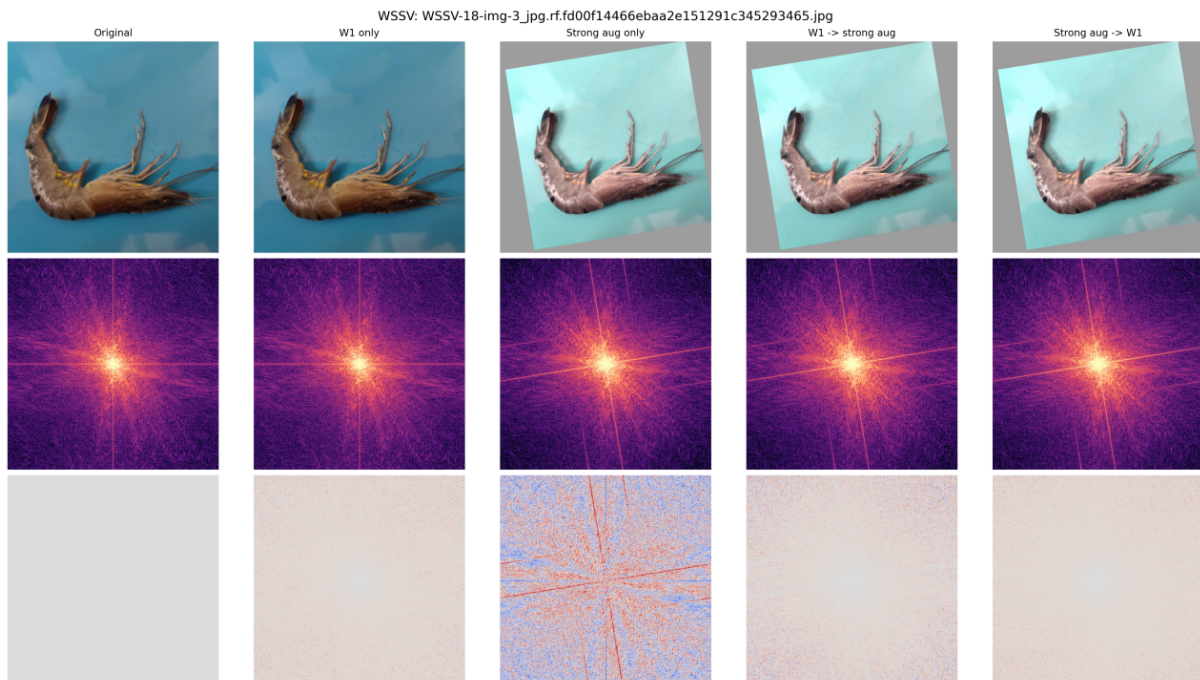


Fig. 4.12. Representative matched five-path construction used by the paired W1-augmentation spectrum study. The first row shows the spatial images, the second row their log-power spectra, and the third row signed spectral differences relative to the matched base path. The figure illustrates the measurement protocol; it is not a model-ranking figure.

The study produced 288 paired observations for each global image metric and 216 observations for lesion-region metrics because Healthy images contain no lesion mask. Recorded measurements included normalized low-, mid-, and high-band spectral-energy share, radial mean power, mean gradient magnitude, Laplacian variance, lesion-region gradient, background gradient, and clipping fraction. These measurements quantify changes in the image distribution and frequency content; they do not themselves retrain the full segmentation model or establish a causal relationship between spectrum changes and mAP.

4.5 Feature-Domain Fourier Refinement

A separate exploratory branch considered Frequency-Domain Detail Enhancement at intermediate feature maps rather than at image input. The operator applies a real-valued Fourier transform to a feature tensor, modulates radial frequency bands with learnable complex gains, reconstructs a detail tensor

by inverse transform, and adds it through a learnable residual gate. P5-only and P3/P4/P5 placements were considered. This branch remains distinct from W1 because it introduces trainable feature-space parameters and changes the network architecture; no unaudited numerical claim is assigned to it here.

4.6 Fourier and Augmentation Order Controls

Four processing-order modes were retained to distinguish deterministic W1 preprocessing from augmentation order. When an offline geometric transform was applied, the same geometry was applied to polygon coordinates. W1 itself required no polygon modification because it preserved image coordinates.

Table 4.10: Fourier-augmentation processing-order protocols

Mode	Training images	Validation/test images	Label handling
Augmentation only	Offline augmentation	Original images	Apply geometric transforms to polygons
W1 then augmentation	W1 preprocessing followed by offline augmentation	W1-preprocessed images	Apply only the geometric stage to polygons
Augmentation then W1	Offline augmentation followed by W1	W1-preprocessed images	Apply geometric stage before W1
Train-only W1	W1-preprocessed images	Original images	W1 requires no polygon modification

In the strict order experiments, online YOLO augmentation and the hidden Albumentations hook were disabled so that the generated image set fully determined transform order. The paired spectrum study in Section 4.4 is diagnostic and should not be conflated with retraining every processing order.

4.7 Segmentation Corruption and Runtime Protocol

For segmentation corruption testing, the recorded order was original evaluation image → synthetic corruption → Fourier preprocessing → model inference. Full-test and disease-labeled subsets were evaluated separately so that localization quality and Healthy-negative behavior were not conflated.

Fourier preprocessing time was measured separately from model inference time. The preprocessing timer covered decoding after the common input stage, channel-wise transform execution, clipping, and reconstruction; repeated model inference was timed after warm-up. Effective pipeline time was summed only when both measurements referred to the same hardware and image set. GPU-side Fourier timing is not used as evidence of Android speed.

5. Evaluation Metrics and Decision Rules

5.1 Classification Metrics

For K classes and N samples, classification accuracy was

$$Accuracy = \frac{\sum_{k=1}^K TP_k}{N}. \quad (4.44)$$

Classwise precision, recall, and F1 were

$$Precision_k = \frac{TP_k}{TP_k + FP_k}, \quad (4.45)$$

$$Recall_k = \frac{TP_k}{TP_k + FN_k}, \quad (4.46)$$

$$F1_k = \frac{2Precision_k Recall_k}{Precision_k + Recall_k}. \quad (4.47)$$

Macro-F1 gave equal weight to every class:

$$Macro - F1 = \frac{1}{K} \sum_{k=1}^K F1_k. \quad (4.48)$$

Cohen's kappa was

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (4.49)$$

where p_o was observed agreement and p_e was agreement expected from the empirical class marginals. Macro-F1 was the primary predictive ranking criterion; accuracy and kappa provided complementary views. Parameters, model size, latency, and FPS were reported as efficiency measurements rather than substitutes for predictive quality.

5.2 Instance-Segmentation Metrics

For predicted mask P and ground-truth mask T , mask intersection over union was

$$IoU(P, T) = \frac{|P \cap T|}{|P \cup T|}. \quad (4.50)$$

Mask precision and recall followed the matched-instance definitions across confidence thresholds. Average precision integrated the precision-recall curve for a class. mAP_{50} averaged class AP at IoU 0.50, whereas $mAP_{50:95}$ averaged AP across IoU thresholds 0.50 to 0.95 in increments of 0.05.

Full-test evaluation included every image in the designated test manifest. Labeled-only evaluation restricted the calculation to images with at least one BG or WSSV ground-truth instance. When a held-out healthy subset was available, the healthy false-positive rate was the fraction of healthy images producing at least one retained disease prediction. Disease missed-image rate was the fraction of diseased images producing no retained prediction. Mask-count mean absolute error was

$$MAE_{count} = \frac{1}{N} \sum_{i=1}^N |\hat{n}_i - n_i|. \quad (4.51)$$

A project-defined HScore combined localization and diagnostic penalties:

$$HScore = mAP_{50}^{\text{labeled}} - 0.05 MAE_{count} - 0.15 MissRate_{\text{disease}} - 0.10 FPR_{\text{healthy}}. \quad (4.52)$$

HScore was an internal decision criterion, not a standardized external benchmark. Its coefficients were fixed before comparing the reported attention configurations.

5.3 Confidence, NMS, and Checkpoint Rules

Ultralytics validation mAP used the framework's validation confidence behavior, recorded as 0.001 for the evaluated runs. Count, missed-image, and Healthy-negative diagnostics used confidence 0.25, class-aware non-maximum suppression with IoU 0.70, and a maximum of 300 detections. These diagnostic thresholds were not substituted into the framework's AP integration.

Segmentation comparisons used best.pt selected by the run's validation fitness. last.pt was excluded from primary comparisons unless explicitly used for convergence diagnosis. Classification candidate screening used the historical test-side metric described in Section 2.2, whereas the selected classifier's training checkpoint was retained through validation-side selection. Table 4.11 consolidates the decision rules.

Table 4.11: Evaluation thresholds and checkpoint rules

Evaluation component	Recorded rule	Interpretation boundary
Classification ranking	Macro-F1 primary; accuracy and kappa complementary	Historical screening used recorded test-side Macro-F1 and is disclosed as such
Segmentation AP	Framework validation confidence behavior, recorded as 0.001	Used for precision-recall integration, not count diagnostics
Count/miss/Healthy diagnostics	confidence 0.25; class-aware NMS IoU 0.70; max detections 300	Fixed operating point for image-level diagnostic measures
Checkpoint	best.pt from validation fitness	last.pt excluded from primary comparisons
Full versus labeled-only	Evaluate both the designated complete set and positive-mask subset	Prevents negative-image behavior from being hidden by positive-only scores
HScore	Equation (4.52) with fixed coefficients	Internal selection aid, not an external benchmark

Table 4.11 prevents the AP confidence behavior from being conflated with the fixed operating point used for count, miss, and Healthy false-positive diagnostics.

6. Mobile Runtime and Real-World Acquisition Protocol

6.1 Common Android Application and Conditional Two-Stage Execution

The runtime-feasibility experiment was Android-only. All three devices ran one common application build and the same two TensorFlow Lite artifacts under the configured four-thread CPU execution path. The classifier was always invoked first. A Healthy classification output or a below-threshold result terminated the pipeline after classification. An above-threshold BG, WSSV, or WSSV_BG output dispatched the instance-segmentation stage. This conditional control flow is part of the runtime protocol and explains why the measured application workload differs across prediction branches.

The common application build was treated as an experimental control, not merely as a deployment convenience. Keeping the application-side code, model-loading logic, preprocessing path, output class order, and conditional branch rule fixed across devices reduces the number of software differences that could otherwise contaminate a runtime-feasibility comparison. The Android experiment therefore changes the physical device and the photographed workload while retaining one application implementation and one pair of exported model artifacts.

The execution path is explicitly sequential. Stage 1 always prepares the classification input and executes the classifier. Only after that output is available can the application decide whether Stage 2 is required. A Healthy result or a result below the configured disease threshold terminates the model pipeline after classification. An above-threshold BG, WSSV, or WSSV_BG result invokes the segmentation artifact. The measured application workload is therefore branch dependent by construction. A run that stops after classification does not perform the same amount of neural-network work as a run that reaches segmentation.

This branch dependence is the main reason device-level runtime means are not interpreted as pure classifier or pure segmentation latency. The runtime log represents the behavior of the conditional application path used in practice. If one device happens to process a larger proportion of segmentation-triggered images, its mean application time can increase even if the underlying hardware is not intrinsically slower. Later device summaries must therefore be read together with the number and type of branches executed, and the study does not convert those means into a strict hardware ranking.

The two exported artifacts also retain different input and precision requirements. The classifier consumes 224 by 224 RGB input in FP32 TFLite form and emits scores in the fixed Healthy, BG, WSSV, WSSV_BG order. The segmentation stage consumes 640 by 640 RGB input in FP16 TFLite form and predicts BG and WSSV instances. The application is responsible for invoking the appropriate preprocessing and output interpretation for each stage. Keeping these interfaces explicit prevents the mobile protocol from being described as if one monolithic model directly produced both the image-level class and the disease-region masks.

CPU execution was selected as the retained common path. Four CPU threads were configured and the GPU delegate was disabled in the recorded application configuration. This decision does not imply that GPU, NNAPI, or other delegates are inferior; those alternatives simply were not the shared execution condition used for the retained three-device study. A common CPU path provides a more uniform implementation contract across the available phones and keeps the reported observation tied to a specific, reproducible runtime setting.

The runtime logs intentionally leave ground-truth fields empty. The mobile photographs were not accompanied by veterinary or laboratory labels, so a predicted class cannot be converted into a field-accuracy claim. The purpose of the application run is to verify that real photographs can pass through the deployed conditional pipeline and produce the expected application behavior on the tested Android devices. Predictive accuracy remains supported by the controlled dataset experiments in Chapter VI, while the mobile branch contributes operational feasibility evidence under clearly bounded conditions.

Table 4.12: Common on-device application and model configuration

Item	Recorded configuration	Methodological role
Application build	Same Android application build on all three devices	Holds application-side implementation constant across device runs
Classification artifact	yolo26m_asl_ldam_simam_dcfr_fp32.tflite; FP32; 224×224 RGB; output order Healthy, BG, WSSV, WSSV_BG	Always executes first and determines whether the second stage is called
Segmentation artifact	yolo11n_seg_float16.tflite; FP16; 640×640 RGB; BG and WSSV instance-mask classes; 5.65 MB	Executes only for an above-threshold disease branch
CPU execution	Four configured CPU threads; GPU delegate disabled in the retained application configuration	Defines the shared on-device execution path
Branch rule	Healthy or below threshold $\rightarrow$ stop after classification; above-threshold BG/WSSV/WSSV_BG $\rightarrow$ classification + segmentation	Separates classification-only and segmentation-triggered runtime paths
Ground-truth fields	ground_truth_label and is_correct left empty	Prevents runtime logs from being misused as field-accuracy evidence

The final classifier parameter count has been audited as 10,886,467. The released FP32 TFLite classifier artifact, yolo26m_asl_ldam_simam_dcfr_fp32.tflite, has an audited serialized size of 41,443,690 bytes. These values are not borrowed from the plain YOLO26m-cls reference row. For segmentation, the retained final architecture is YOLO11n-seg with CA $\rightarrow$ SimAM at P3/P4/P5, with 2,854,718 parameters and approximately 9.7 GFLOPs; notebook-GPU timing for that model is kept separate from this Android protocol.

6.2 Three-Camera Real-World Acquisition Set

The operational acquisition archive contains 21 usable real-world shrimp photographs collected with the three team-owned Android phones. The retained images intentionally span practical capture variation, including different viewpoints and orientations, close-range framing, uneven or low illumination, camera softness or blur, and cluttered backgrounds. One unusably dark photograph was excluded because the shrimp could not be inspected reliably. The archive is an operational input set rather than an expert-labeled disease benchmark; no veterinary or laboratory diagnosis was assigned to these photographs.

The three phones provide distinct camera sources, but the retained acquisition metadata do not support a controlled resolution-specific comparison. Camera megapixel or image-resolution effects are therefore not claimed or inferred.

The three-camera acquisition set was designed to exercise the deployed application with images that were not copied from the training or evaluation datasets. Its methodological value is therefore different from that of the fixed test partitions. The 21 retained photographs expose the application to ordinary capture variation that occurs when users frame a shrimp with a phone rather than when a curated image is read from a benchmark folder. Variations in viewpoint, orientation, distance, illumination, focus softness, blur, and background clutter were preserved rather than normalized into one laboratory-style capture condition.

This diversity is intentionally operational rather than statistically representative. The photographs do not define a population estimate for farms, disease prevalence, camera brands, or environmental conditions. They are a small feasibility set collected with the three available team-owned Android devices. Their purpose is to test whether the application accepts and processes non-dataset images across several practical capture situations, not to establish an external clinical benchmark.

The three phones provide genuinely different camera sources, but the experiment does not isolate camera resolution as an independent variable. The retained metadata do not contain a controlled protocol in which the same scene was photographed at matched distances, exposure settings, compression settings, and known resolutions while all other factors were held constant. Consequently, neither megapixel count nor image resolution is used to explain runtime or prediction behavior. The report records multiple camera sources while explicitly stopping short of a resolution-effect claim.

One unusably dark photograph was excluded because the shrimp could not be inspected reliably in the retained image. This exclusion is reported because a feasibility archive should distinguish a missing or unusable capture from a successful operational input. The remaining 21 photographs are the usable set for the mobile observations. Their inclusion does not imply that every photograph has a verified disease condition, nor does visual plausibility substitute for expert diagnosis.

The lack of veterinary, laboratory, or independently qualified disease ground truth is a deliberate interpretation boundary. The application may output Healthy, BG, WSSV, or WSSV_BG and may conditionally invoke segmentation, but those outputs are not scored as correct or incorrect for this acquisition archive. Accordingly, the study does not calculate field accuracy, sensitivity, specificity, or prevalence from these photographs. A future field-validation study would require prospective image capture together with independently reviewed disease labels and a protocol that defines how uncertain or mixed cases are adjudicated.

The acquisition set also should not be conflated with the synthetic corruption experiments. Controlled noise, blur, low-light, and related transformations provide repeatable stress conditions on labeled dataset images, whereas the phone photographs combine naturally occurring changes without controlled severity or ground truth. The two branches answer different questions: synthetic corruptions measure model sensitivity under defined transformations, while the three-camera archive demonstrates that the deployed application can be exercised with heterogeneous real-world inputs. Section 6.3 then records the device configurations and sampling procedure used for those operational runs.

6.3 Runtime Device Configuration and Sampling

Table 4.13: Android runtime device configuration

Device	OS / processor as recorded	CPU	Reported memory / storage	GPU
vivo Y100	Android 15; Snapdragon 685, reported up to 2.8 GHz	8 cores	16 GB reported configuration; 128 GB storage	Not recorded
Samsung A32	Android 13; MediaTek Helio G80, 2.00 GHz	8 cores	6 + 6 GB reported configuration; 128 GB storage	Not recorded
POCO M7 Pro 5G	Android 16; MediaTek Dimensity 7025 Ultra	8 cores	8 + 4 GB reported configuration; 256 GB storage	Not recorded

Memory entries are preserved as the device-reported/configured values available during the experiment. Values written as “6 + 6” or “8 + 4” are not reinterpreted here as a single physical-RAM quantity, and missing GPU identifiers are reported as not recorded rather than guessed.

Each member recorded 21 application runs on one device using manually uploaded real-world shrimp photographs. Exact image identity and ordering were not standardized across devices; equal run count, common app/model configuration, and runtime feasibility were the experimental priorities. Accordingly, device-level means are not designed as a controlled image-by-image hardware ranking. They must later be interpreted together with branch composition because disease-triggered runs execute the second model while Healthy or below-threshold runs do not.

6.4 Interpretation Boundary for the Mobile Experiment

The predicted class is used only to determine which application branch executed; it is not treated as a verified disease label. No field accuracy, sensitivity, specificity, or prevalence is computed because the photographed shrimp did not receive veterinary, laboratory, or other expert disease ground truth. The recorded time is interpreted as application-side runtime feasibility unless a narrower timer scope is explicitly demonstrated. iOS was not tested because no compatible iPhone/iOS device was available, so the study supports Android feasibility only and does not claim iOS compatibility.

7. Experimental Environment and Reproducibility

Training hardware, notebook-GPU efficiency timing, Fourier preprocessing timing, and Android application runtime are retained as distinct provenance categories. The primary training environment was a Kaggle Linux notebook with two NVIDIA Tesla T4 GPUs available; this environment must not be conflated with the separate notebook timings used for model-efficiency screens or with the Android runtime logs.

Table 4.14: Training, analysis, and reproducibility configuration

Component	Recorded configuration	Preserved evidence
Primary training platform	Kaggle Linux; dual NVIDIA Tesla T4 environment; 31.348 GiB RAM	Notebook/export, environment capture, run directory
Final classification training	YOLO26m-cls + ASL-LDAM + SimAM-DCFR; 224×224 ; nominal 70/15/15; AdamW; LR 0.00125; cosine; final LR factor 0.01; max 30 epochs; patience 15; batch 32; 4 workers; AMP; seed 42	Run arguments, class mapping, manifest, checkpoint, prediction outputs

Table 4.14 (continued)

Component	Recorded configuration	Preserved evidence
Classification software	Python 3.12.13; PyTorch 2.10.0+cu128; TorchVision 0.25.0+cu128; CUDA 12.8; cuDNN 91002; Ultralytics 8.4.75	Package/version capture and training log
Final segmentation method	YOLO11n-seg + CA → SimAM at P3/P4/P5; 640 × 640 RGB; nominal 80/10/10 grouped protocol; stronger augmentation; 100 configured / 100 executed epochs; yolo11n-seg.pt initialization, 594/594 tensors; 2,854,718 parameters; ~9.7 GFLOPs	Dataset/model YAML, run arguments, checkpoint, CSV
Selected attention software	Ultralytics 8.4.67; PyTorch 2.11.0+cu128 for retained CA → SimAM run	Run-specific package metadata and logs
Reproducibility identity	Class mappings, split manifests, specimen keys, dataset/model YAML, seeds, checkpoints, training CSV, predictions, corruption definitions, spectrum scripts, figure-generation scripts	Machine-readable artifacts retained with the corresponding experiment
Timing provenance	Candidate-screen GPU, attention notebook GPU, Fourier preprocessing/model timing, and Android application runtime are separate scopes	Timing logs preserved with hardware, task, and measured-stage context

The selected segmentation run does not receive an invented optimizer specification in this chapter because the retained evidence does not independently verify that field. The final classifier parameter count is audited as 10,886,467; the released FP32 TFLite classifier artifact has an audited serialized size of 41,443,690 bytes. These audited values are not borrowed from the unmodified YOLO26m candidate entry and preserve artifact-level traceability rather than filling unsupported metadata.

8. Methodological Validity and Scope Boundaries

The evidence is primarily fixed-seed and does not establish multi-seed statistical significance. Classification candidate screening historically inspected test-partition Macro-F1, limiting the independence of that partition for model-family selection. The approximately 15M-parameter deployment preference is an engineering constraint for this project rather than a universal definition of a lightweight classifier.

Segmentation masks were created under a project annotation protocol without independent expert boundary adjudication. Healthy images are known empty-label negatives, not missing annotations. Filename-derived specimen grouping prevents documented repeated-specimen overlap only where filenames expose compatible specimen identifiers; the 303 unmatched-name images in the expanded inventory are therefore stratified at image level, so the complete expanded split is not described as fully specimen-leakage-free.

The broader attention-family comparison is configuration-level evidence rather than a strict one-factor ablation because augmentation, epoch budgets, early stopping, initialization, software versions, placement, and auxiliary objectives differ. The separate CA → SimAM placement study controls the insertion-site question more narrowly, but it does not remove the protocol differences among the other attention families.

The paired Fourier study quantifies how the recorded transforms alter image and spectral distributions. It does not retrain the complete segmentation model under every processing order and therefore does not by itself prove a causal mechanism linking a spectral change to mAP. Controlled synthetic corruptions and Fourier timing similarly remain protocol-specific diagnostics rather than field-validation evidence.

The three-camera mobile photographs demonstrate operational capture diversity but have no expert disease ground truth. The Android runtime experiment uses equal run counts but not a standardized image-by-image workload across devices, so it is a runtime-feasibility study rather than a strict hardware benchmark. No iOS device was tested. The complete system remains a preliminary image-based screening prototype and does not replace laboratory testing or veterinary assessment.

References

1. L. Yang, R.-Y. Zhang, L. Li, and X. Xie, "SimAM: A simple, parameter-free attention module for convolutional neural networks," in Proc. 38th Int. Conf. Machine Learning, 2021, pp. 11863-11874.
2. K. Cao, C. Wei, A. Gaidon, N. Arechiga, and T. Ma, "Learning imbalanced datasets with label-distribution-aware margin loss," in Advances in Neural Information Processing Systems 32, 2019.
3. E. Ben-Baruch, T. Ridnik, N. Zamir, A. Noy, I. Friedman, M. Protter, and L. Zelnik-Manor, "Asymmetric loss for multi-label classification," in Proc. IEEE/CVF Int. Conf. Computer Vision, 2021, pp. 82-91.